

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Yoga Andreas Hutajulu - 5024241021

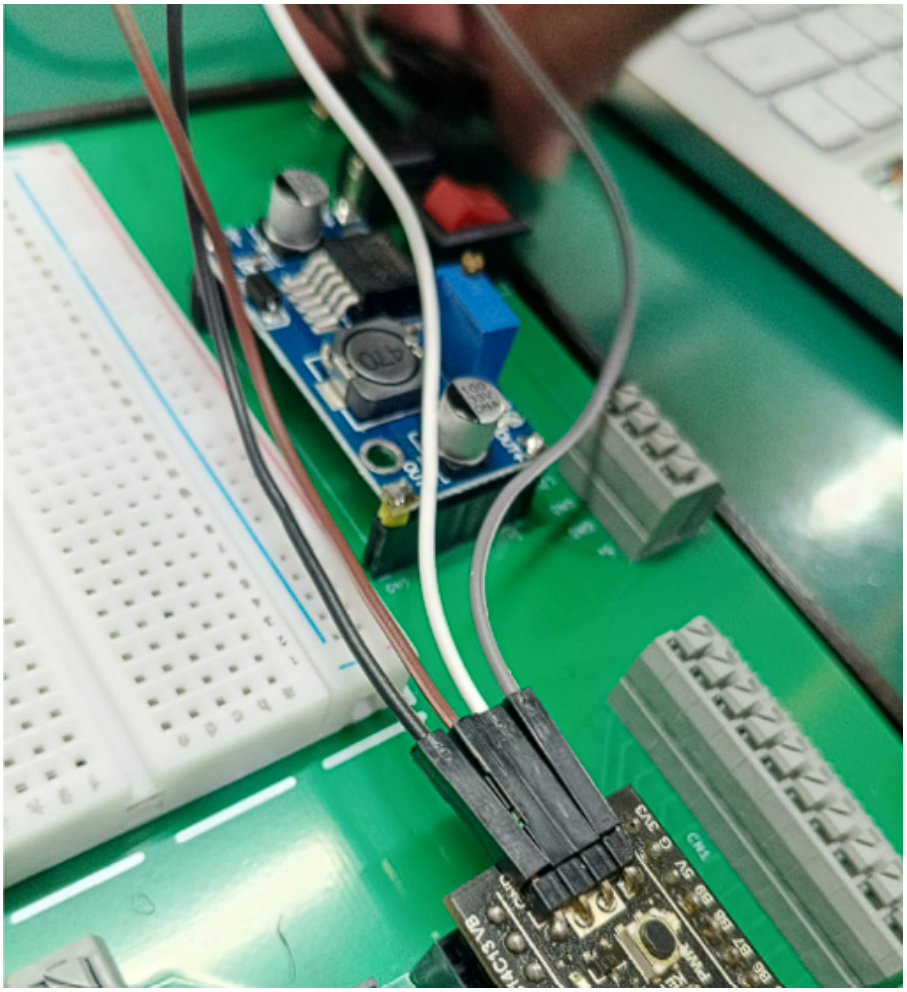
2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Pastikan project Modul 1 telah dibuat dan konfigurasi `platformio.ini` telah disesuaikan menggunakan board STM32 Blackpill F401CC dengan framework Arduino dan protokol upload ST-Link.

2. Hubungkan ST-Link ke board STM32 Blackpill menggunakan kabel dengan mencocokkan pin SWDIO, SWCLK, GND, dan 3.3V. Pastikan ST-Link menggunakan tegangan 3.3V.



Gambar 1: Board Dihubungkan ke Komputer

3. Hubungkan ST-Link ke komputer melalui USB dan tunggu hingga perangkat terdeteksi.

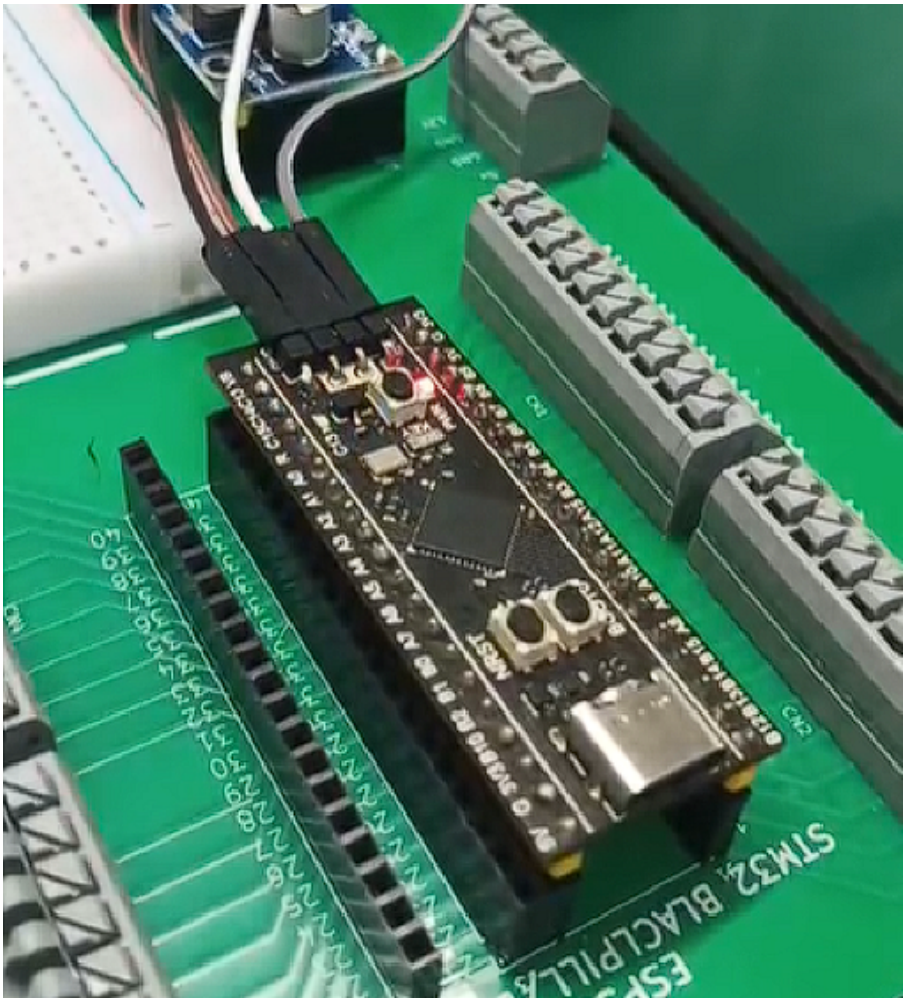
4. Pastikan proses refresh konfigurasi PlatformIO telah selesai sebelum melakukan proses build dan upload.

5. Tuliskan program Blink pada file `src/main.cpp` menggunakan pin PC13 sebagai LED onboard.

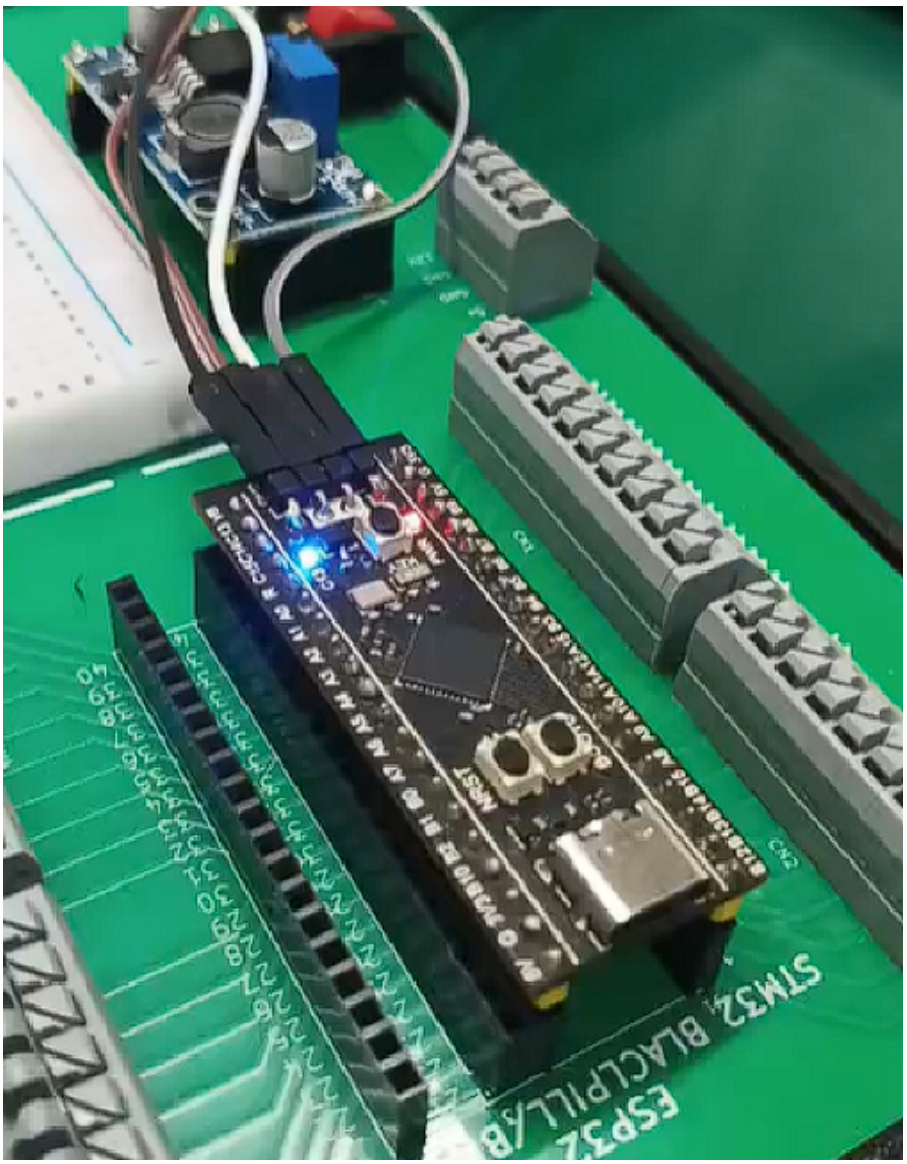
6. Lakukan proses Build dan Upload untuk mengirimkan program ke board STM32 Blackpill.

7. Amati LED onboard pada pin PC13. Pastikan LED menyala selama 500 ms dan mati selama 500 ms secara berulang.

Hasil Percobaan:



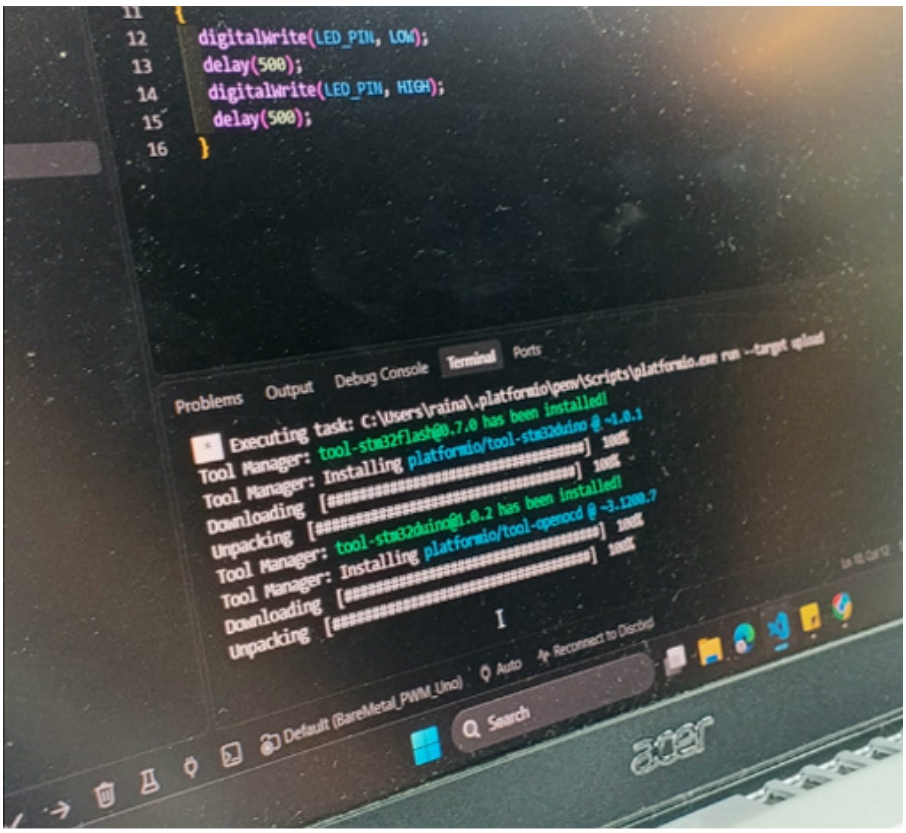
Gambar 2: Hasil Percobaan 1: Pengenalan STM32 Blackpill (Blink), mati



Gambar 3: Hasil Percobaan 1: Pengenalan STM32 Blackpill (Blink), nyala

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

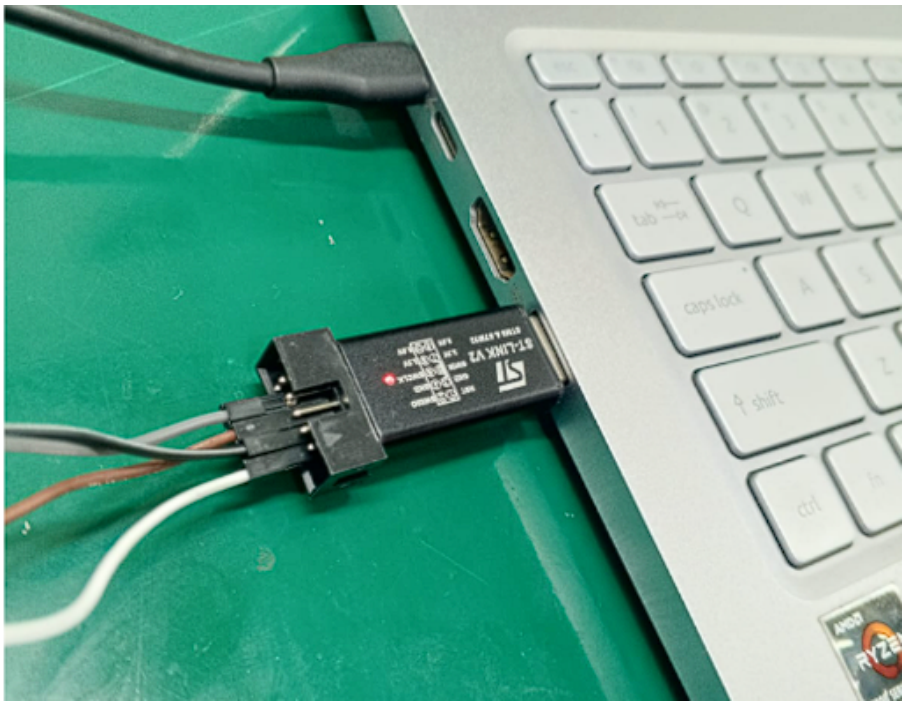
1. Ubah konfigurasi `platformio.ini` menggunakan board ESP32 DevKit dengan framework ESP-IDF.



Gambar 4: Konfigurasi Platformio

2. Tunggu hingga proses refresh konfigurasi PlatformIO selesai sebelum melanjutkan proses build dan upload.

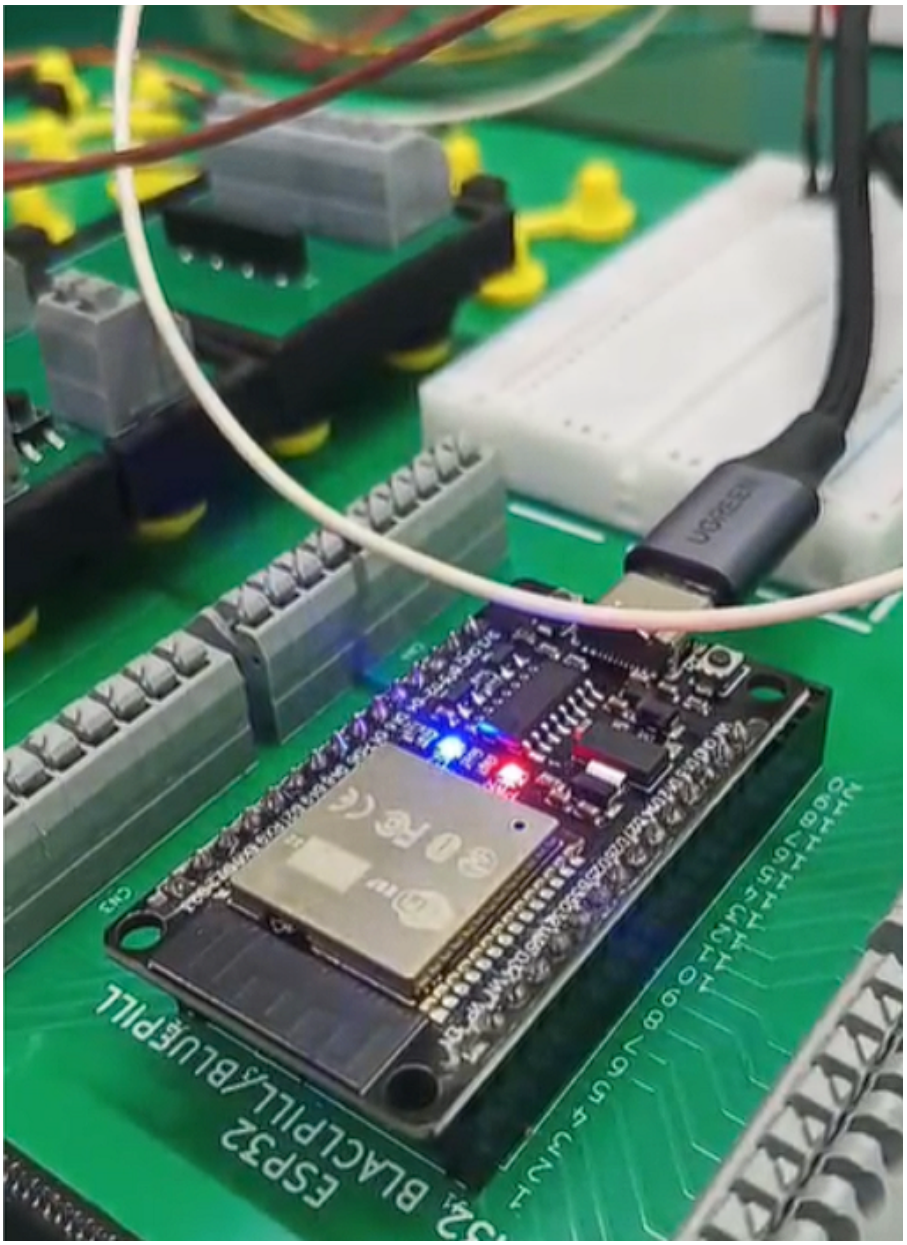
3. Hubungkan board ESP32 ke komputer menggunakan kabel USB dan pastikan perangkat telah terdeteksi.



Gambar 5: board dihubungkan ke komputer

4. Hapus file `src/main.cpp`, kemudian buat file baru bernama `src/main.c`.
5. Tuliskan program Blink menggunakan framework ESP-IDF dengan memanfaatkan fungsi GPIO dan FreeRTOS.
6. Lakukan proses Build dan Upload untuk mengirimkan program ke board ESP32.
7. Amati LED onboard ESP32 untuk memastikan LED menyala dan mati secara bergantian dengan interval 500 ms.
8. Analisis perbedaan proses pemrograman menggunakan framework ESP-IDF dan Arduino.

Hasil Percobaan:

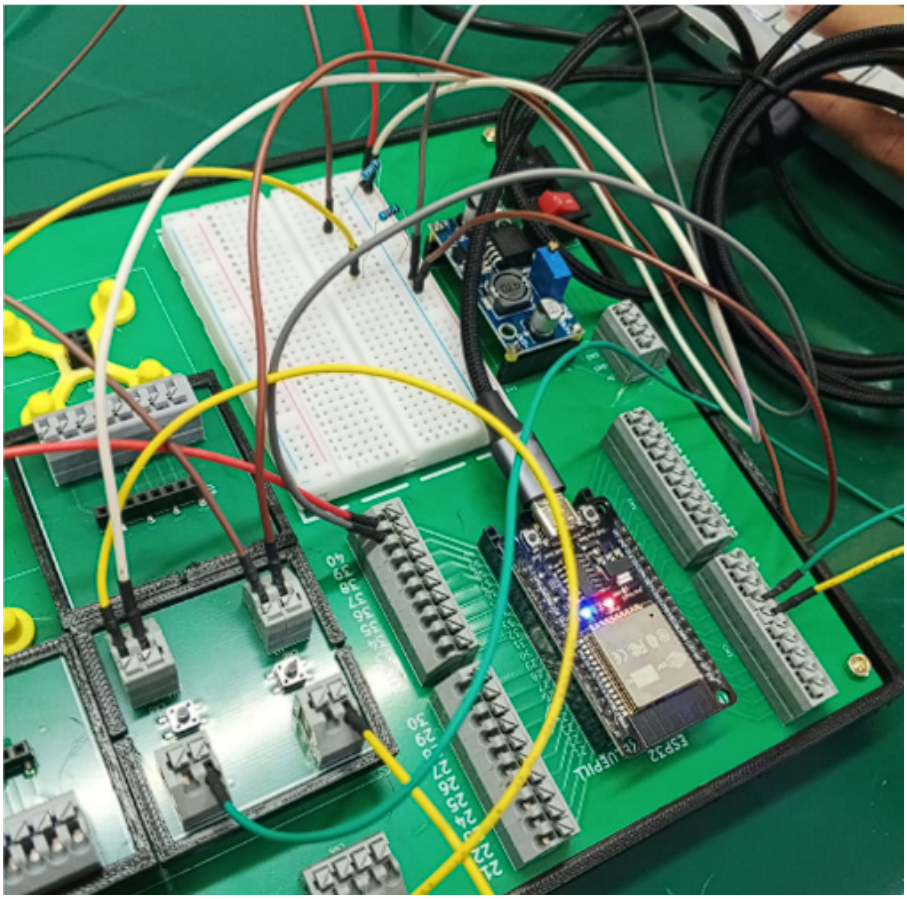


Gambar 6: Hasil Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF

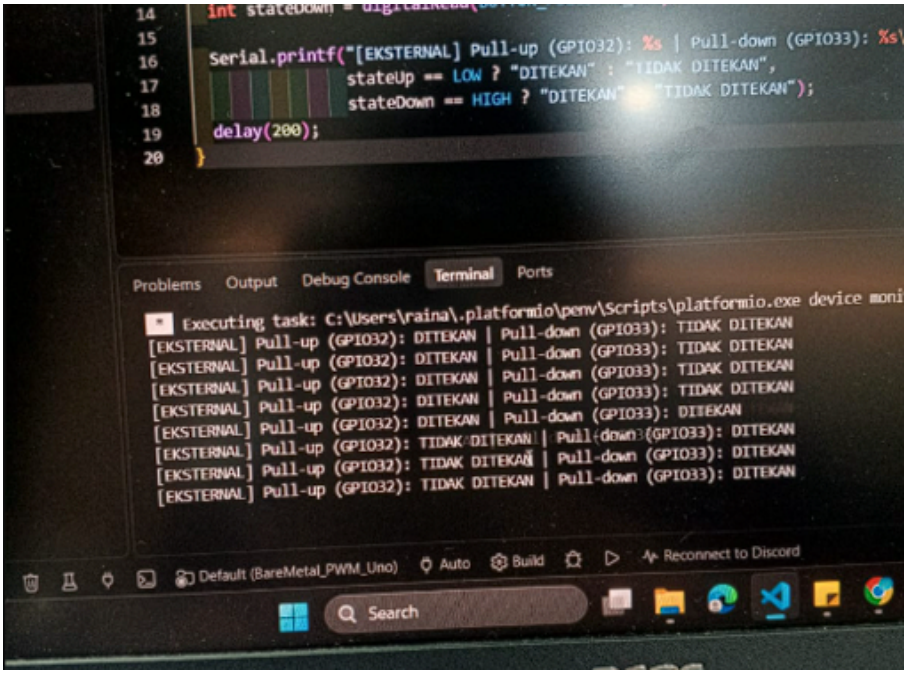
1.3 Percobaan 3: Pull-up dan Pull-down Eksternal dan Internal

1. Ubah konfigurasi `platformio.ini` menggunakan board ESP32 DevKit dengan framework Arduino. Hapus file `src/main.c`, kemudian buat kembali file `src/main.cpp`.
2. Rangkai dua pushbutton pada GPIO 32 dan GPIO 33 sesuai skema rangkaian bagian A dengan menggunakan resistor eksternal 10 kΩ sebagai pull-up dan pull-down.
3. Tuliskan Kode Program A pada file `src/main.cpp` dengan mengatur kedua pin sebagai input menggunakan fungsi `pinMode()`.
4. Lakukan proses Build dan Upload, kemudian buka Serial Monitor dengan baud rate 115200.
5. Tekan kedua pushbutton secara bergantian dan amati perubahan kondisi yang ditampilkan pada Serial Monitor.
6. Lepaskan kedua resistor eksternal tanpa mengubah posisi pushbutton sesuai skema rangkaian bagian B.
7. Ganti program sebelumnya dengan Kode Program B yang menggunakan konfigurasi `INPUT_PULLUP` dan `INPUT_PULLDOWN`.
8. Lakukan kembali proses Build dan Upload, kemudian amati hasil pembacaan kedua pushbutton melalui Serial Monitor.
9. Bandingkan hasil pembacaan antara penggunaan resistor eksternal dan internal. Pastikan kondisi pull-up bernilai HIGH saat tombol tidak ditekan dan LOW saat ditekan, sedangkan pull-down menunjukkan kondisi sebaliknya.

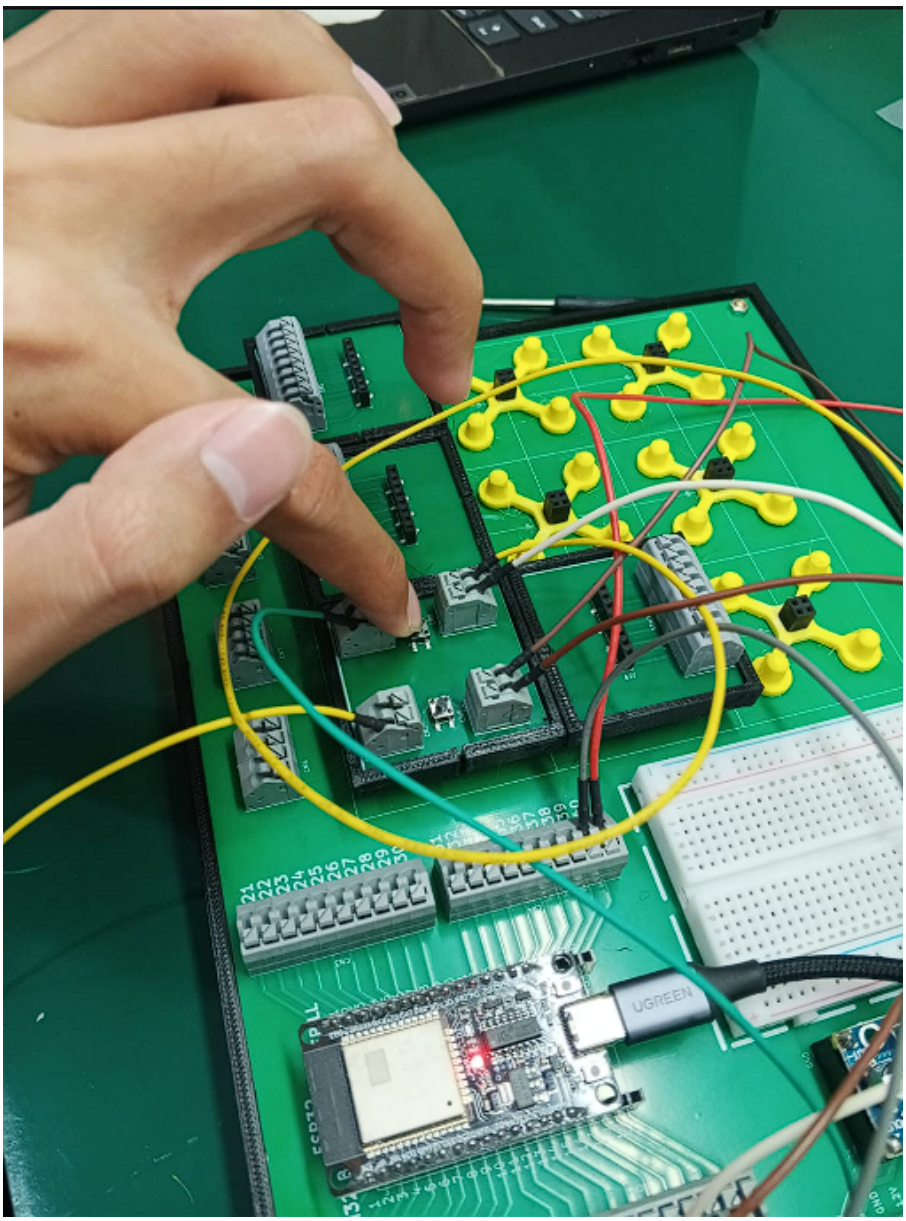
Hasil Percobaan:



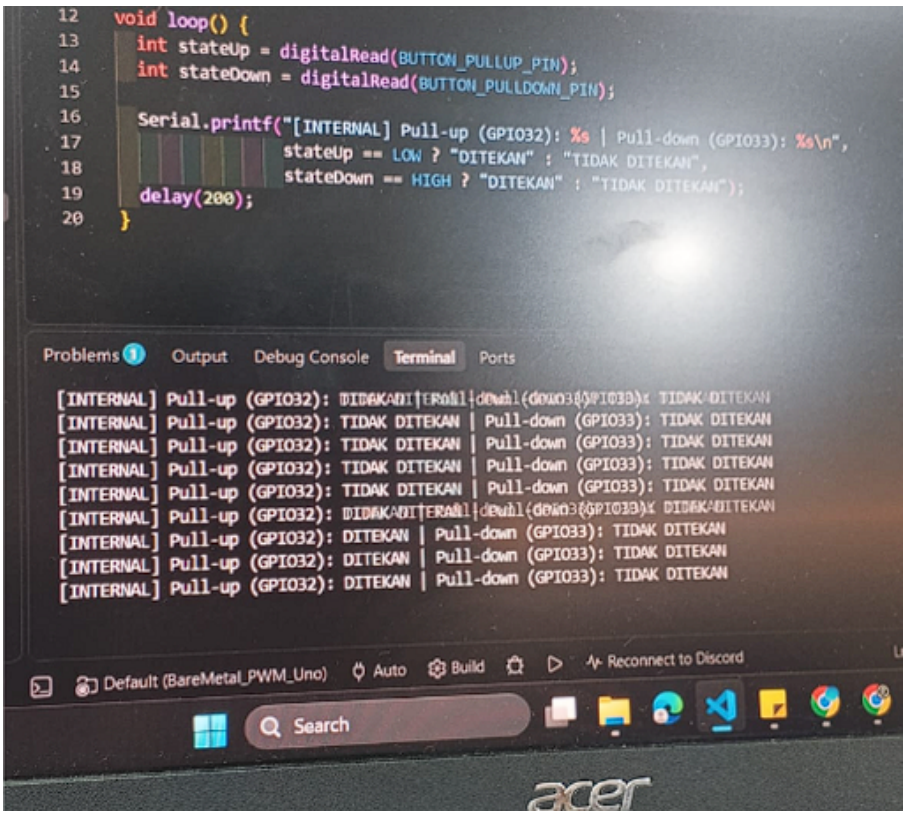
Gambar 7: Rangkaian Percobaan 3 Eksternal



Gambar 8: Hasil Percobaan 3: Pull-up dan Pull-down Eksternal



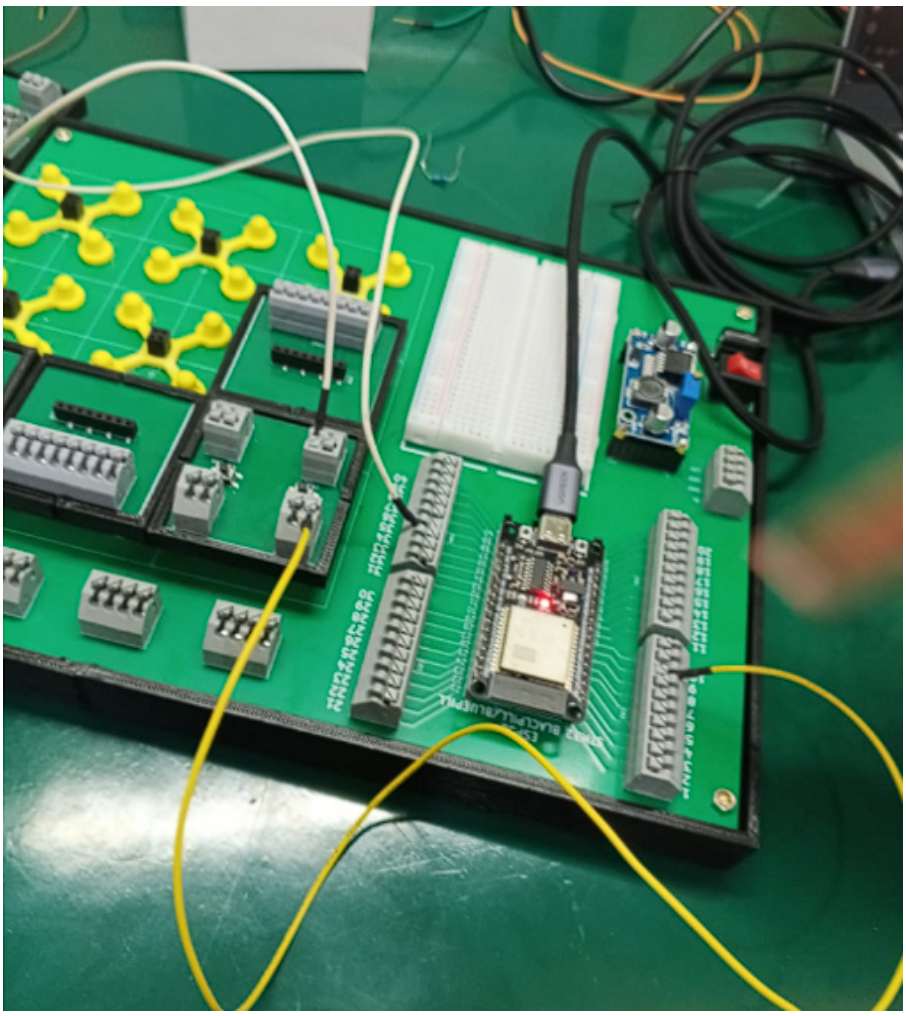
Gambar 9: Rangkaian Percobaan 3 Internal



Gambar 10: Hasil Percobaan 3: Pull-up dan Pull-down Internal

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO

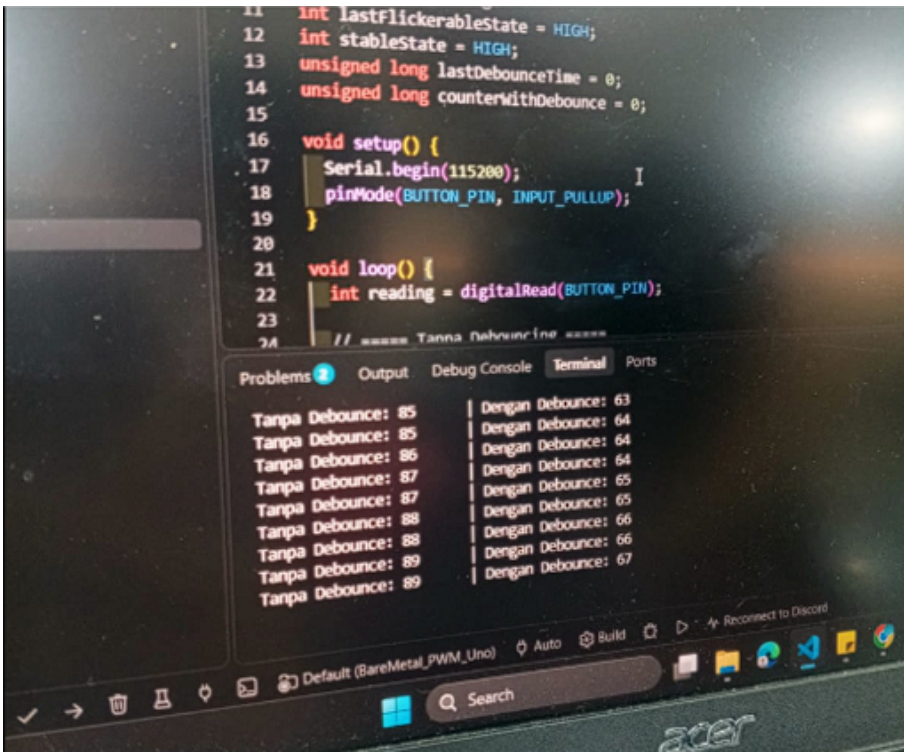
1. Rangkai satu pushbutton tactile pada GPIO 25 ESP32 dengan menghubungkan salah satu kaki tombol ke GPIO 25 dan kaki lainnya ke GND.



Gambar 11: Rangkaian Percoban 4

2. Tuliskan program debouncing pada file `src/main.cpp` dengan mengatur pin GPIO 25 sebagai input menggunakan `INPUT_PULLUP`.
3. Lakukan proses Build dan Upload, kemudian buka Serial Monitor dengan baud rate 115200.
4. Tekan tombol satu kali dan amati perbedaan nilai penghitung `counterNoDebounce` dan `counterWithDebounce`
5. Ulangi penekanan tombol dengan kecepatan yang berbeda, kemudian amati kemungkinan terjadinya penambahan nilai penghitung lebih dari satu kali pada kondisi tanpa debouncing.
6. Ubah nilai `DEBOUNCE_DELAY` menjadi 10 ms, kemudian lakukan kembali proses Build dan Upload. Amati respons tombol melalui Serial Monitor.
7. Ubah nilai `DEBOUNCE_DELAY` menjadi 200 ms, kemudian lakukan kembali proses Build dan Upload.
8. Tekan tombol secara berulang dengan cepat dan amati perubahan nilai penghitung.
9. Bandingkan hasil pengujian menggunakan nilai debounce 10 ms, 50 ms, dan 200 ms untuk mengetahui pengaruh durasi debounce terhadap kestabilan pembacaan dan respons tombol.

Hasil Percobaan:

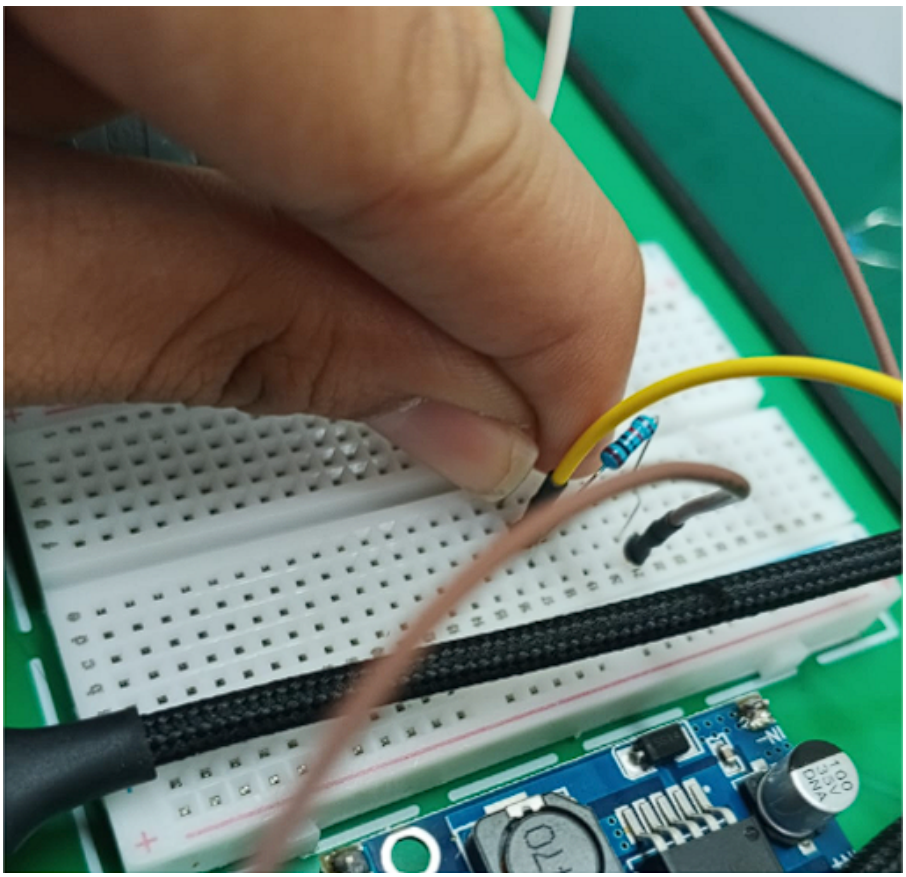


Gambar 12: Hasil Percobaan 4: Debouncing pada Input Tombol GPIO

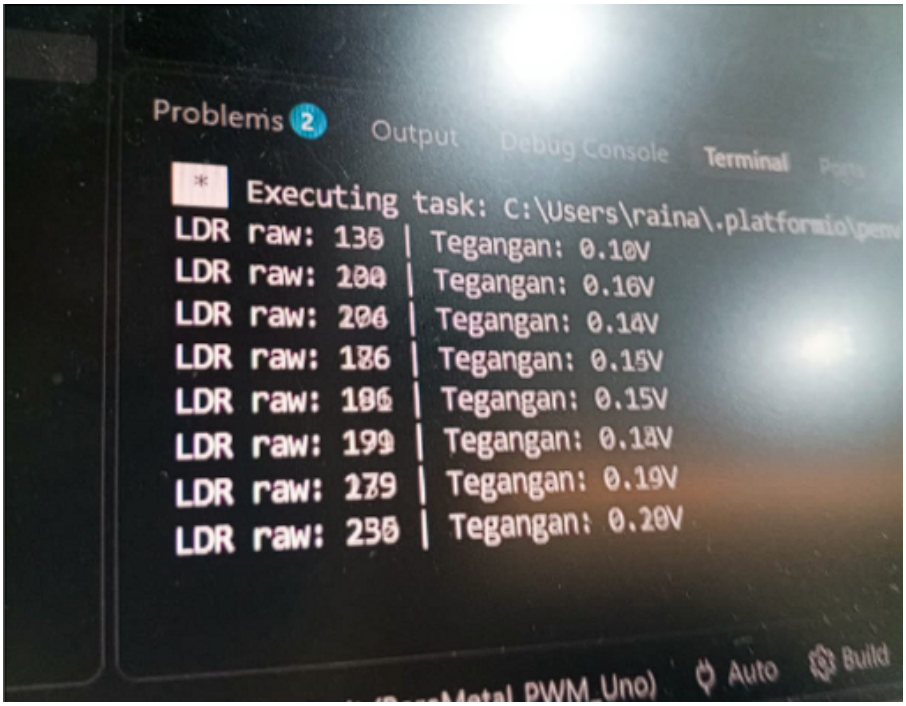
1.5 Percobaan 5: Akses GPIO Analog dengan Pembacaan ADC menggunakan LDR

1. Rangkai komponen LDR dan resistor 10 kΩ sebagai pembagi tegangan sesuai skema rangkaian.
2. Hubungkan titik tengah antara LDR dan resistor ke GPIO 34 pada board ESP32.
3. Tuliskan program pembacaan ADC pada file `src/main.cpp` dengan menggunakan fungsi `analogRead()`.
4. Lakukan proses Build dan Upload, kemudian buka Serial Monitor dengan baud rate 115200.
5. Amati nilai ADC mentah dan tegangan yang ditampilkan pada Serial Monitor.
6. Tutup permukaan LDR menggunakan tangan, kemudian berikan pencahayaan menggunakan lampu.
7. Amati perubahan nilai ADC dan tegangan yang dihasilkan berdasarkan perbedaan intensitas cahaya yang diterima LDR.
8. Analisis perubahan nilai pembacaan analog serta perbedaannya dengan pembacaan sinyal digital yang hanya memiliki kondisi HIGH dan LOW.

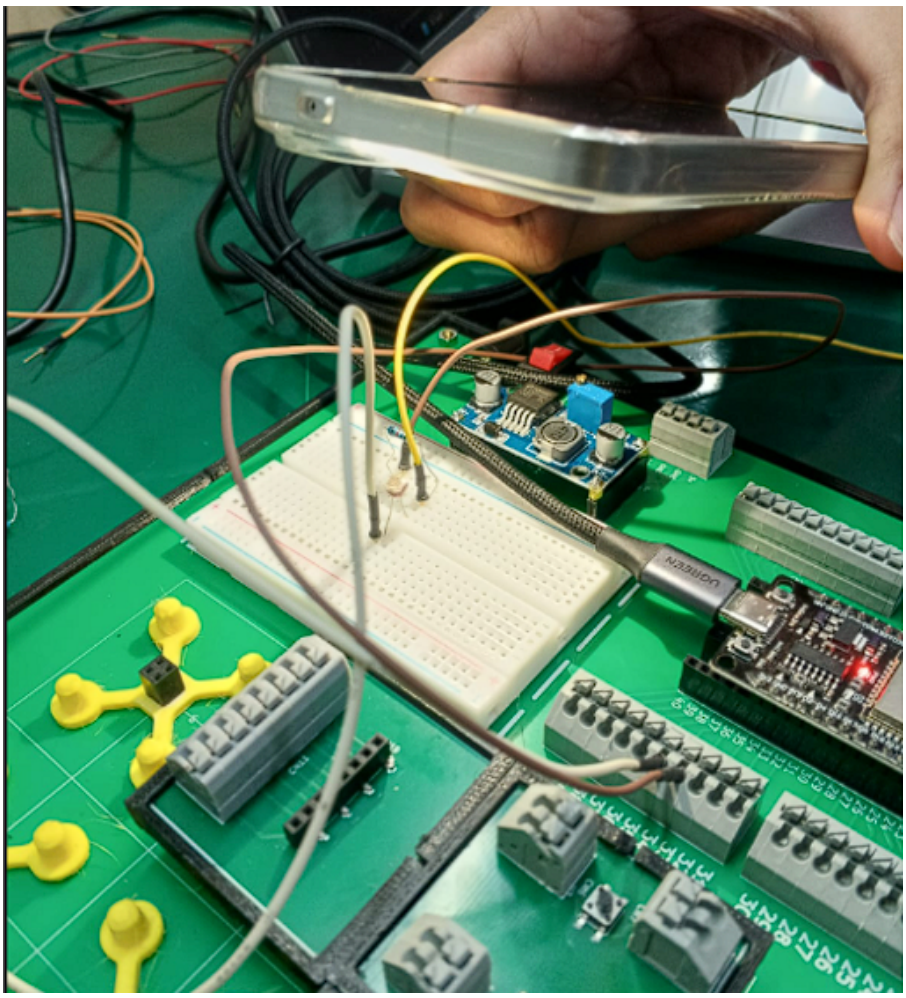
Hasil Percobaan:



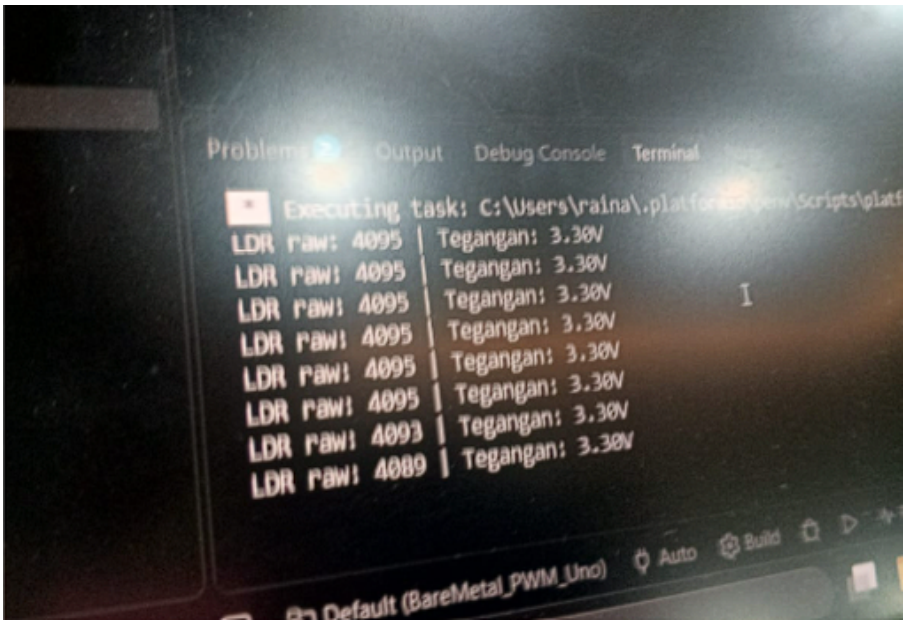
Gambar 13: LDR Ditutup



Gambar 14: Hasil Percobaan 5: Pembacaan ADC LDR ditutup



Gambar 15: LDR Disinari



Gambar 16: Hasil Percobaan 5: Pembacaan ADC LDR Disinari

2 Analisis Percobaan

2.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Pada percobaan pertama, dilakukan pengujian board STM32 Blackpill menggunakan program Blink dengan framework Arduino. Berdasarkan hasil pengamatan, LED onboard pada pin PC13 dapat menyala dan mati secara bergantian dengan interval 500 ms. Hal ini menunjukkan bahwa program berhasil diunggah ke board melalui ST-Link dan pin GPIO dapat digunakan sebagai output digital.

LED pada board STM32 Blackpill menggunakan konfigurasi aktif LOW, sehingga LED menyala ketika pin diberikan logika LOW dan mati ketika diberikan logika HIGH. Hasil tersebut sesuai dengan teori pengendalian GPIO digital. Apabila LED tidak menyala atau program tidak berjalan, kemungkinan penyebabnya adalah kesalahan koneksi ST-Link, konfigurasi pin yang tidak sesuai, atau proses upload yang belum berhasil. Dengan demikian, percobaan ini berhasil memperkenalkan proses pemrograman dan pengendalian GPIO pada STM32 Blackpill.

2.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Pada percobaan kedua, dilakukan pengujian board ESP32 menggunakan framework ESP-IDF untuk menjalankan program Blink. Berdasarkan hasil pengamatan, LED onboard dapat menyala dan mati secara bergantian dengan interval 500 ms. Hal ini menunjukkan bahwa program berhasil dijalankan dan konfigurasi GPIO sebagai output telah bekerja dengan baik.

Berbeda dengan framework Arduino yang menggunakan fungsi `setup()` dan `loop()`, ESP-IDF menggunakan fungsi `app_main()` sebagai titik awal program. Pengaturan GPIO juga dilakukan secara lebih langsung menggunakan fungsi seperti `gpio_set_direction()` dan `gpio_set_level()`. Selain itu, penggunaan `vTaskDelay()` menunjukkan bahwa pengaturan waktu dilakukan melalui sistem FreeRTOS. Apabila LED tidak bekerja, kemungkinan terdapat kesalahan konfigurasi framework, pemilihan pin, atau proses upload. Percobaan ini menunjukkan bahwa ESP-IDF memberikan pengaturan perangkat keras yang lebih terperinci dibandingkan framework Arduino.

2.3 Percobaan 3: Pull-up dan Pull-down Eksternal dan Internal

Pada percobaan ketiga, dilakukan perbandingan pembacaan pushbutton menggunakan resistor pull-up dan pull-down eksternal maupun internal pada ESP32. Berdasarkan hasil pengamatan, kedua metode dapat menghasilkan pembacaan logika yang sama selama rangkaian dan konfigurasi pin telah dipasang dengan benar. Pada pull-up, pin membaca HIGH ketika tombol tidak ditekan dan LOW ketika ditekan. Sebaliknya, pada pull-down, pin membaca LOW ketika tombol tidak ditekan dan HIGH ketika ditekan.

Hasil tersebut sesuai dengan teori bahwa resistor pull-up berfungsi menjaga kondisi awal pin pada logika HIGH, sedangkan pull-down menjaga kondisi awal pada logika LOW. Perbedaannya terletak pada penggunaan resistor, yaitu metode eksternal memerlukan resistor tambahan pada rangkaian, sedangkan metode internal memanfaatkan resistor yang sudah tersedia di dalam ESP32. Jika hasil pembacaan tidak sesuai, kemungkinan penyebabnya adalah kesalahan pemasangan kaki pushbutton, resistor yang belum dilepas saat menggunakan mode internal, atau koneksi rangkaian yang kurang tepat. Dengan demikian, percobaan ini menunjukkan bahwa kedua metode dapat digunakan untuk menjaga kestabilan kondisi input digital.

2.4 Percobaan 4: Debouncing pada Input Tombol GPIO

Pada percobaan keempat, dilakukan pengujian debouncing untuk mengamati pengaruh getaran mekanis pada pushbutton. Berdasarkan hasil pengamatan, penghitung tanpa debouncing berpotensi bertambah lebih dari satu kali dalam satu tekanan tombol. Hal ini terjadi karena kontak mekanis pada tombol dapat menghasilkan beberapa perubahan sinyal dalam waktu singkat sebelum benar-benar stabil. Sementara itu, penghitung dengan debouncing hanya bertambah setelah perubahan sinyal melewati waktu tertentu sesuai nilai `DEBOUNCE_DELAY`.

Perbedaan nilai debounce 10 ms, 50 ms, dan 200 ms juga memengaruhi respons tombol. Nilai yang lebih besar dapat mengurangi pengaruh bouncing, tetapi menyebabkan respons tombol menjadi lebih lambat. Sebaliknya, nilai yang terlalu kecil berpotensi membuat getaran tombol masih terdeteksi sebagai beberapa tekanan. Faktor seperti kondisi fisik tombol dan kecepatan penekanan juga dapat memengaruhi hasil pengamatan. Percobaan ini menunjukkan bahwa debouncing diperlukan untuk menghasilkan pembacaan tombol yang lebih stabil dan akurat.

2.5 Percobaan 5: Akses GPIO Analog dengan Pembacaan ADC menggunakan LDR

Pada percobaan kelima, dilakukan pembacaan nilai analog menggunakan sensor LDR yang dihubungkan dengan pin GPIO 34 ESP32 melalui rangkaian pembagi tegangan. Berdasarkan hasil pengamatan, nilai ADC dan tegangan yang ditampilkan pada Serial Monitor mengalami perubahan sesuai dengan intensitas cahaya yang diterima LDR. Ketika kondisi cahaya berubah, resistansi LDR juga berubah sehingga tegangan pada titik tengah pembagi tegangan ikut mengalami perubahan.

Nilai ADC yang diperoleh berada pada rentang 0–4095 karena ESP32 menggunakan ADC dengan resolusi 12-bit. Nilai tersebut kemudian dikonversi menjadi perkiraan tegangan menggunakan perhitungan berdasarkan tegangan referensi 3,3 V. Berbeda dengan input digital yang hanya membaca kondisi HIGH atau LOW, input analog memungkinkan pembacaan berbagai tingkat tegangan. Perbedaan hasil pembacaan dapat dipengaruhi oleh intensitas cahaya, posisi sumber cahaya, karakteristik LDR, dan ketepatan pemasangan resistor. Dengan demikian, percobaan ini menunjukkan bahwa ADC dapat digunakan untuk membaca perubahan kondisi lingkungan melalui sensor analog.

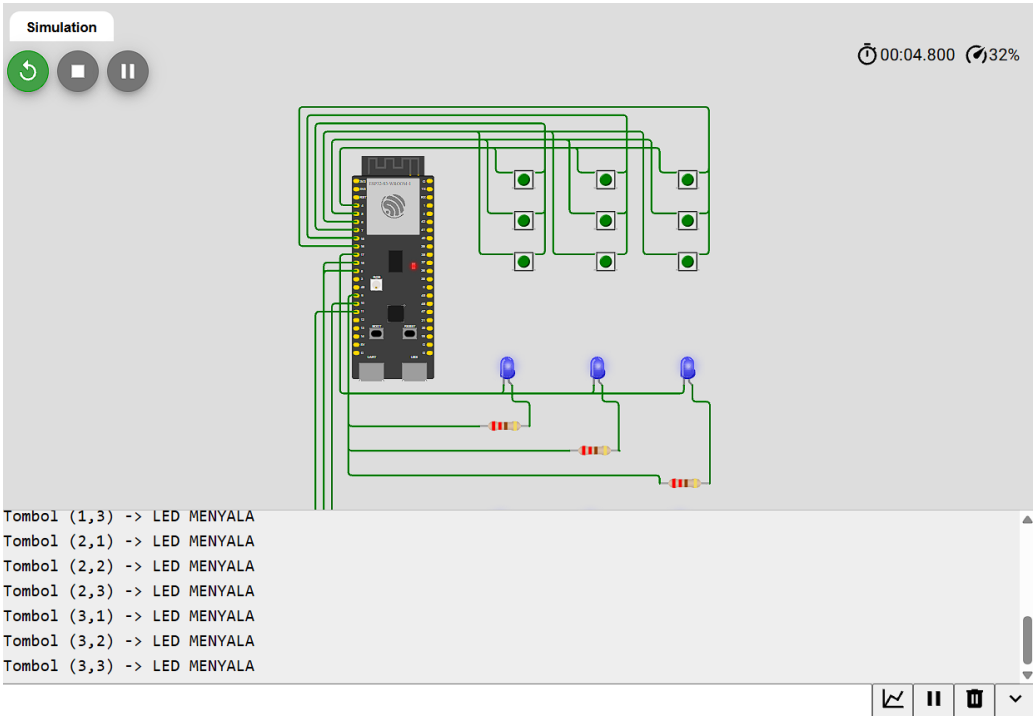
3 Tugas Modul

3.1 Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3

Pada tugas modul ini, dibuat simulasi rangkaian Matrix Pushbutton 3x3 yang digunakan untuk mengendalikan Matrix LED 3x3 menggunakan mikrokontroler ESP32-S3. Setiap tombol dihubungkan berdasarkan posisi baris dan kolomnya dengan LED pada posisi yang sama. Sebagai contoh, ketika tombol pada baris 1 kolom 1 ditekan, LED pada baris 1 kolom 1 akan menyala. Program juga menerapkan metode debouncing pada setiap tombol agar perubahan status LED hanya terjadi setelah kondisi pembacaan tombol stabil.

3.1.1 Hasil Simulasi

Berikut merupakan hasil simulasi rangkaian Matrix Pushbutton 3x3 yang digunakan untuk mengendalikan Matrix LED 3x3.



Gambar 17: Rangkaian Simulasi Matrix Pushbutton 3x3 dan Matrix LED 3x3

Gambar berikut menunjukkan source code untuk simulasi.

```
1
2   #include <Arduino.h>
3
4   const int buttonRows[3] = {4, 5, 6};
5   const int buttonCols[3] = {7, 15, 16};
6
7   const int ledRows[3] = {17, 18, 8};
8   const int ledCols[3] = {9, 10, 11};
9
10  bool ledState[3][3] = {false};
11
12  bool lastRawState[3][3] = {false};
13  bool stableState[3][3] = {false};
14
15  unsigned long lastDebounceTime[3][3] = {0};
16
17  const unsigned long DEBOUNCE_DELAY = 50;
18
19  void setup() {
20
21    Serial.begin(115200);
22
23    for (int i = 0; i < 3; i++) {
24      pinMode(buttonRows[i], INPUT);
25      pinMode(buttonCols[i], INPUT_PULLUP);
26    }
27
28    for (int i = 0; i < 3; i++) {
29      pinMode(ledRows[i], OUTPUT);
30      pinMode(ledCols[i], OUTPUT);
31
32      digitalWrite(ledRows[i], LOW);
33      digitalWrite(ledCols[i], HIGH);
34    }
```

Gambar 18: Source code Matrix LED 3x3

3.2 Analisis Percobaan

1. **Alasan debouncing tetap diperlukan pada rangkaian Matrix Pushbutton 3x3**

   Debouncing tetap diperlukan karena tombol mekanis dapat mengalami bouncing, yaitu kondisi ketika kontak di dalam tombol mengalami getaran atau perubahan sambungan sesaat saat ditekan maupun dilepas. Akibatnya, mikrokontroler dapat membaca satu kali penekanan sebagai beberapa perubahan sinyal HIGH dan LOW dalam waktu singkat. Meskipun biasanya hanya satu tombol yang ditekan pada satu waktu, bouncing tetap dapat menyebabkan perubahan status LED yang tidak sesuai. Oleh karena itu, debouncing digunakan untuk memastikan bahwa kondisi tombol telah stabil selama waktu tertentu sebelum perubahan status LED dilakukan.

2. **Analisis konfigurasi pull-up pada pembacaan kolom tombol**

   Pada rangkaian ini digunakan konfigurasi internal pull-up pada pin kolom tombol dengan meng-

atur pin sebagai `INPUT_PULLUP`. Ketika tombol tidak ditekan, pin kolom akan berada pada kondisi HIGH karena terhubung secara internal ke tegangan positif melalui resistor pull-up. Sebaliknya, ketika tombol ditekan, pin kolom akan terhubung ke jalur baris yang sedang diatur LOW sehingga pembacaan berubah menjadi LOW. Konfigurasi ini diperlukan agar pin memiliki kondisi logika yang jelas. Jika pin dibiarkan floating atau tidak terhubung ke kondisi HIGH maupun LOW yang pasti, pembacaan dapat berubah-ubah akibat gangguan listrik di sekitarnya. Hal tersebut dapat menyebabkan mikrokontroler mendeteksi penekanan tombol yang sebenarnya tidak terjadi.

3. **Perbedaan pembacaan digital input dan analog input menggunakan ADC**

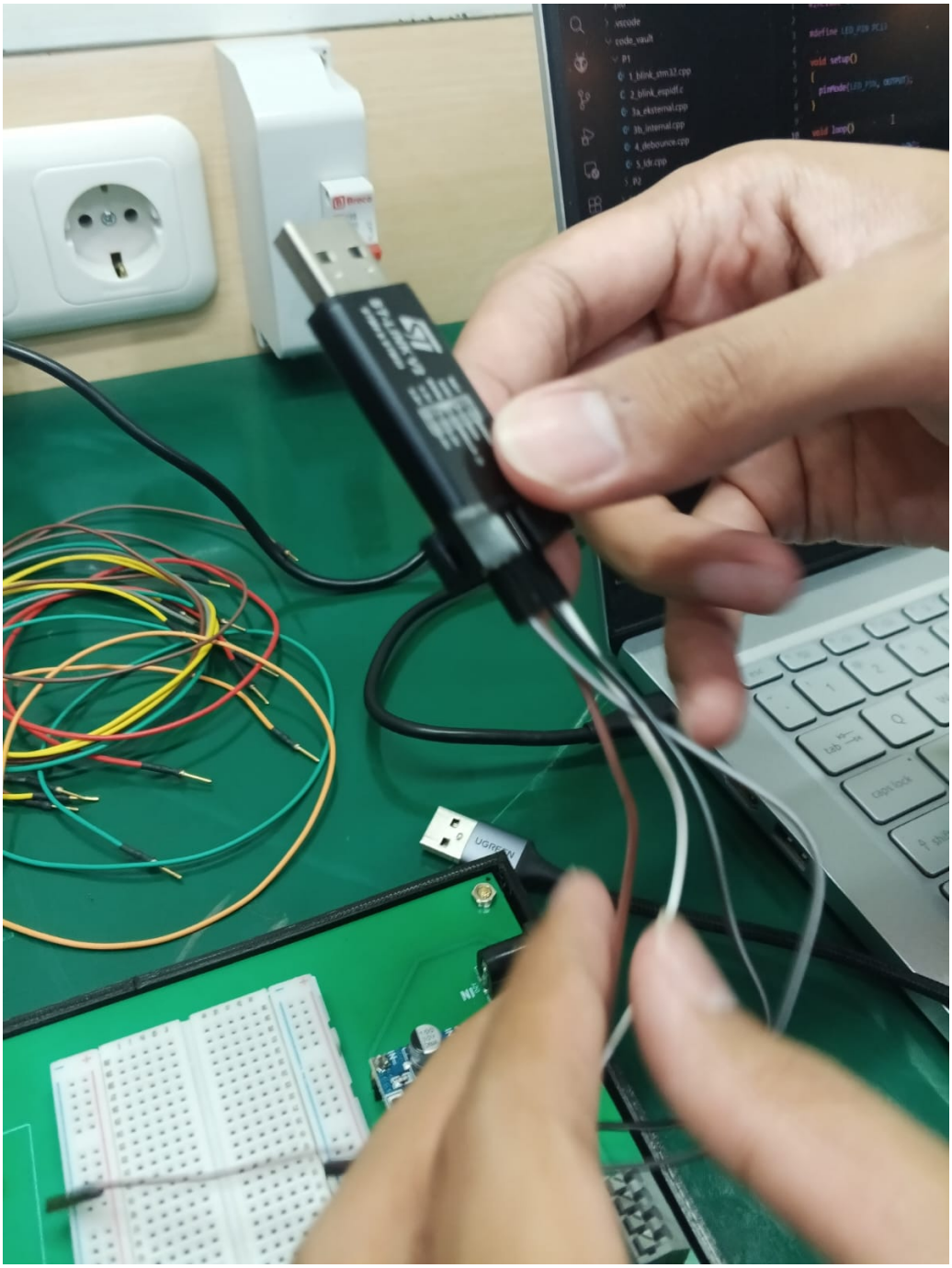
   Digital input hanya mengenali dua kondisi logika, yaitu HIGH dan LOW. Pada rangkaian tombol, kondisi tersebut digunakan untuk mengetahui apakah tombol sedang ditekan atau tidak. Sementara itu, analog input menggunakan ADC (*Analog to Digital Converter*) untuk mengubah tegangan analog menjadi nilai digital yang dapat diproses oleh mikrokontroler. ADC dapat merepresentasikan lebih dari dua kondisi karena mampu mengukur beberapa tingkat tegangan dalam rentang tertentu. Sebagai contoh, pada ADC 12-bit ESP32-S3, hasil pembacaan memiliki rentang nilai 0–4095. Ketika LDR digunakan sebagai pengganti tombol, perubahan intensitas cahaya akan mengubah resistansi LDR dan tegangan pada rangkaian pembagi tegangan. Perubahan tegangan tersebut kemudian dibaca oleh ADC sehingga mikrokontroler dapat mengetahui tingkat intensitas cahaya, bukan hanya kondisi menyala atau mati.

4 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa ESP32-S3 mampu membaca input dari Matrix Pushbutton 3x3 dan mengendalikan Matrix LED 3x3 sesuai dengan posisi tombol yang ditekan. Penerapan debouncing diperlukan untuk mencegah pembacaan tombol yang tidak stabil akibat bouncing. Selain itu, penggunaan pull-up membuat kondisi input lebih stabil dan menghindari pembacaan yang tidak menentu. Praktikum ini juga memberikan pemahaman mengenai perbedaan input digital dan analog serta cara kerja ADC dalam membaca perubahan tegangan.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 19: Lampiran saat praktikum